Atividade Avaliativa A2 – Parte 2:

Felipe Rochoel Costa RA:2679698
Vinicius Gabriel Giordani Barbosa RA:2669650

Repositório GitHub: https://github.com/Vinegabriel/ES-A2-Parte2-SistemaAgendamento

SEÇÃO 1: Projeto Inicial

História Escolhida para Implementação

História 2 (Prioridade Alta):

Como aluno, eu quero visualizar horários livres e reservar um atendimento de 30 minutos na disciplina desejada para garantir meu suporte nos estudos.

1. Mapeamento de Classes e Responsabilidades

Para atender à História 2 sem dependência de entradas manuais (*input*) durante a execução dos testes automatizados, a lógica do sistema foi encapsulada nas seguintes classes em Python:

1. Classe Horario:

- **Responsabilidade:** Representar a entidade de um bloco individual de atendimento de 30 minutos e manter o seu estado.
- **Atributos:**
  - ***id_horario***: Identificador único do bloco de atendimento.
  - ***disciplina***: Nome da matéria (ex: Cálculo, Física, POO, ED).
  - ***monitor***: Nome do monitor responsável.
  - ***dia_semana:*** Dia da semana do atendimento (ex: "Segunda-feira").
  - ***data_atendimento:*** Data do atendimento, usada para identificar a semana da reserva.
  - ***hora_inicio***: Horário de início do bloco.
  - ***status***: Estado atual do bloco ("Livre", "Reservado" ou "Cancelado").
  - ***aluno_reservado***: Nome do aluno que realizou a reserva (inicialmente None).

2. Classe SistemaMonitoria:

- **Responsabilidade:** Gerenciar a coleção de horários em memória e aplicar todas as regras de negócio de consulta e reserva.
- **Métodos Principais:**
  - ***cadastrar_horario***(id_horario, disciplina, monitor, dia_semana, hora_inicio, data_atendimento): Adiciona um novo bloco disponível ao sistema.
  - ***listar_horarios_livres***(disciplina): Filtra e retorna apenas os blocos com status "Livre" para a disciplina pesquisada.
  - ***reservar_horario***(id_horario, aluno, disciplina): Valida as regras de negócio antes de confirmar o agendamento:
    1. Verifica se o bloco informado existe e se seu status é "Livre".
    2. Contabiliza quantas reservas o mesmo aluno já possui na mesma disciplina e semana ISO, calculada a partir da data do atendimento.
    3. Se o horário estiver livre e o aluno tiver menos de 2 reservas na disciplina, atualiza o status para "Reservado" e vincula o nome do aluno.
    4. Caso o horário esteja ocupado/cancelado ou o aluno já tenha atingido o limite de 2 reservas semanais, lança uma exceção do tipo ValueError.

2. Planejamento dos Testes de Aceitação (unittest)

Os testes automatizados foram implementados no arquivo *test_agendamento.py* utilizando a biblioteca unittest. Cada um dos 4 critérios de aceitação da História 2 é coberto por um teste específico:

- **Critério 1 (Listagem de horários livres):**
  test_listar_horarios_livres_sucesso
  - *Verificação:* Garante que a consulta filtre apenas blocos com status "Livre" na disciplina solicitada.
- **Critério 2 (Reserva de horário vago - Sucesso):** *test_reservar_horario_sucesso*
  - *Verificação:* Confirma que, ao reservar um bloco livre, o status altera para "Reservado" e vincula o nome do aluno.
- **Critério 3 (Reserva de horário ocupado - Falha/Exceção):** *test_reservar_horario_ja_ocupado_falha*

    - *Verificação:* Garante que tentar agendar um bloco "Reservado" lança exceção ValueError sem alterar a reserva anterior.
- **Critério 4 (Limite semanal de 2 reservas - Falha/Exceção):** *test_limite_maximo_reservas_semanais_falha*
    - *Verificação:* Garante que o sistema dispara exceção ValueError quando um aluno tenta realizar a 3ª reserva na mesma disciplina.

Seção 2:Relatório de Execução (README)

1. Instruções para Execução do Sistema e Testes

- Linguagem / Versão: Python 3.15 ou superior, sem dependências de bibliotecas externas.
- Arquivos Feitos: *monitoria.py*, contém a lógica de negócio e *test_agendamento.py*, contém os testes de aceitação.
- Estrutura dos Arquivos: Todos os arquivos de código-fonte estão localizados no diretório código/, abra o terminal na raiz da pasta.
- Comando para execução dos testes automatizados:

```
Python
python -m unittest test_agendamento.py
```

2. Comparação: Plano vs Realizado

- 16/10 Análise e Plano: Feito e entregue em sala. Feito por Vinicius e Felipe.
- 17/10 Repositório e Histórias de Usuário: Adiado para o dia 18 para todos os contribuintes participarem do planejamento das histórias de usuário. Feito por Vinicius e Felipe.
- 19/10 Projeto Inicial, Implementação e Testes: Elaboração do Projeto Inicial focado na história 2 e desenvolvimento do código fonte. Devido a dispensa da aula do dia 19, adiantamos o que estava previsto para o dia 20, implementando os testes. Feito por Vinicius e Felipe.
- 21/10 Relatório e Documentação: Feito o relatório de execução e compilação da documentação PDF. Feito por Vinicius e Felipe.
- 22/10 Autoavaliação, Ajustes Finais e Entrega: Identificamos que a validação do Critério 4 precisava de checagem de calendário. Alteramos a classe Horario para receber data_atendimento e atualizamos a reserva com isocalendar(). Adicionamos testes, validamos o .zip e entregamos no Moodle. Feito por Vinicius e Felipe.

3. Atrasos e Mudanças em Relação ao Projeto Inicial

- Atrasos: Não ocorreu nenhum atraso durante o desenvolvimento da Atividade Avaliativa, na verdade conseguimos adiantar um dia de desenvolvimento.
- Mudanças na Arquitetura: O desenvolvimento seguiu o planejamento inicial, apenas mudança na Regra de Negócio (22/10): Inclusão do tratamento de datas com `datetime` para refletir fielmente a restrição semanal estipulada pelo Critério 4, o que exigiu a atualização dos construtores nos testes e na lógica principal, mas todos os outros critérios foram implementados corretamente

SEÇÃO 3: Documentação Técnica

1. Histórias de Usuário

História 1

- **Descrição**: Como monitor, eu quero cadastrar meus horários disponíveis para que os alunos possam reservar atendimento.
- **Prioridade:** Alta (A base do sistema; sem horários ofertados não há como haver reservas).
- **Trecho do E-mail:** *"No início do semestre, cada monitor informa os horários em que pode atender..."*

- **Critérios de Aceitação:**
    1. Dado que o monitor acessa o cadastro, quando informa dia, horário inicial e final, o sistema registra a disponibilidade.
    2. Dado que o monitor salva um período, o sistema divide-o automaticamente em blocos de 30 minutos.
    3. Dado que o monitor tenta salvar horários em branco, o sistema exibe mensagem de erro e impede o salvamento.

História 2 (HISTÓRIA IMPLEMENTADA)

- **Descrição**: Como aluno, eu quero visualizar horários livres e reservar um atendimento de 30 minutos na disciplina desejada para garantir meu suporte nos estudos.
- **Prioridade:** Alta (Funcionalidade central indicada como prioridade principal pela coordenação).
- **Trecho do E-mail:** *"A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento."*

- **Critérios de Aceitação:**

1. Dado que o aluno pesquisa por uma disciplina, o sistema lista apenas os horários com status "Livre".
2. Dado que o aluno confirma a reserva de um horário vago, o sistema altera seu status para "Reservado" e o vincula ao aluno.
3. Dado que o aluno tenta reservar um horário já ocupado, o sistema recusa a operação e lança exceção.
4. Dado que o aluno tenta realizar a 3ª reserva na mesma disciplina e semana, o sistema bloqueia o agendamento informando o limite máximo de 2 reservas.

História 3

- **Descrição**: Como monitor, eu quero poder cancelar um horário cadastrado para evitar desencontros e visitas desnecessárias dos alunos.
- **Prioridade:** Média (Resolve o problema de cancelamentos de última hora mencionado no e-mail).
- **Trecho do E-mail:** *"...mas às vezes precisa cancelar por causa de prova. Os alunos reclamam que só descobrem o cancelamento quando chegam na sala."*

- **Critérios de Aceitação:**
    1. Dado que o monitor remove um horário vago, ele é retirado imediatamente das buscas públicas.
    2. Dado que o monitor cancela um horário reservado, o status muda para "Cancelado" e o aluno associado é notificado.
    3. Dado que um horário foi cancelado, qualquer tentativa futura de reserva por um aluno é bloqueada.

História 4

- **Descrição**: Como coordenação, eu quero gerar um relatório mensal restrito com as horas atendidas por monitor para efetuar o pagamento das bolsas.
- **Prioridade:** Média (Necessária para a gestão financeira do curso e pagamento das bolsas).
- **Trecho do E-mail:** *"A coordenação precisa de um relatório mensal com as horas atendidas por cada monitor, porque isso conta para o pagamento da bolsa. Essas informações de bolsa*

não podem ficar visíveis para todo mundo."

- **Critérios de Aceitação:**
    1. Dado que a coordenação solicita o relatório, o sistema contabiliza o total de horas de atendimentos concluídos no mês por monitor.
    2. Dado que um usuário comum (aluno ou monitor) tenta acessar o relatório de bolsas, o sistema nega o acesso.
    3. Dado que o relatório é gerado, ele exibe o nome do monitor, disciplina e total de horas contabilizadas.

História 5

- **Descrição**: Como professor, eu quero visualizar relatórios com os tópicos de maior dúvida dos alunos para reforçar o conteúdo nas aulas.
- **Prioridade:** Baixa (Recurso analítico de apoio pedagógico que não impede a operação do agendamento).
- **Trecho do E-mail:** *"Os professores das disciplinas gostariam de saber quais conteúdos geram mais dúvidas, para reforçar em aula."*

- **Critérios de Aceitação:**
    1. Dado que o professor acessa o relatório, ele pode filtrar as dúvidas registradas por disciplina e período.
    2. Dado que existam agendamentos sem especificação de tema, o sistema ignora registros vazios na contagem.
    3. Dado que o relatório do professor é exibido, ele não exibe dados referentes a valores ou horas de bolsa dos monitores.

2. Mapeamento dos Testes de Aceitação (História 2)

Critério de Aceitação	**Caso de Teste (unittest)**	**Cenário Coberto**	**Resultado**
Critério 1: Listar horários livres	*test_listar_horarios_livres_sucesso*	Sucesso	Passou
Critério 2: Reservar horário vago	*test_reservar_horario_sucesso*	Sucesso	Passou
Critério 3: Tentar reservar horário ocupado	*test_reservar_horario_ja_ocupado_falha*	Falha / Exceção (*ValueError*)	Passou
Critério 4: Limite semanal de 2 reservas	*test_limite_maximo_reservas_semanais_falha*	Falha / Exceção (*ValueError*)	Passou
Critério 4: Validação em semanas distintas	*test_limite_semanal_nao_bloqueia_reserva_em_outra_semana*	Sucesso	Passou
Regra de Negócio: Validação de Disciplina	*test_reservar_horario_de_outra_disciplina_falha*	Falha / Exceção (*ValueError*)	Passou

- **O que não foi coberto e justificativa:** Os fluxos de relatórios para coordenação/professores e o cancelamento de horários (HUs 1, 3, 4 e 5) não foram cobertos nesta etapa. Conforme as regras da atividade, o escopo de implementação e testes focou 100% na História de Prioridade Alta (História 2), atendendo plenamente a todos os requisitos mínimos exigidos.